

Detalhamento técnico

Sistema de emissão de Notas Fiscais — Angular 19 + ASP.NET Core / C# (.NET 10) + PostgreSQL. Dois microsserviços: Estoque e Faturamento.

Estoque :5081

Faturamento :5082

App :4200

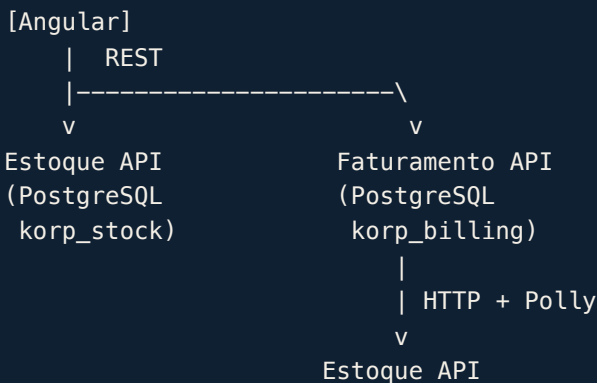
PostgreSQL :5433

1. Visão da solução

O frontend Angular consome duas APIs REST independentes, cada uma com o próprio banco:

- **Korp.Stock.Api** (:5081): cadastro de produtos, consulta de saldo, baixa/estorno com lock de concorrência e modo de falha simulada.
- **Korp.Billing.Api** (:5082): numeração sequencial de notas, itens, impressão, compensação e assistente.

A baixa de estoque **não** ocorre no cadastro da nota. Ocorre só na **impressão**, como pede o enunciado. Se o estoque falhar no meio da impressão, o faturamento estorna as baixas já feitas e a nota permanece **Aberta**.



2. Quais ciclos de vida do Angular foram utilizados

Hook	Onde	Para quê
<code>ngOnInit</code>	Painel, produtos, notas, shell	Dispara cargas HTTP, busca com debounce e <i>health checks</i> .
<code>ngAfterViewInit</code>	Cadastro de produto e editor da nota	Foco no primeiro campo/botão ao abrir a tela.
<code>ngOnChanges</code>	<code>InvoiceItemRowComponent</code>	Quando o produto do item muda, atualiza o saldo visível na linha.
<code>ngOnDestroy</code>	Shell, painel, produtos, editor	Cancela <code>Subscription</code> / <code>takeUntil</code> para não vazar memória nem HTTP.

Os componentes são **standalone**. As rotas de tela usam `loadComponent` (lazy loading).

3. Se foi feito uso da biblioteca RxJS e, em caso afirmativo, como

Sim. O RxJS é o modelo de assincronismo do frontend.

- `switchMap` + `debounceTime` + `distinctUntilChanged` : busca de produtos (cancela a request anterior enquanto a pessoa digita).
- `interval` + `startWith` + `switchMap` + `catchError` : *polling* de saúde das APIs a cada 5s, sem quebrar a UI se uma delas cair.
- `combineLatest` : painel junta produtos, notas e insights numa única atualização.
- `exhaustMap` : botão Imprimir ignora cliques repetidos enquanto a request está em voo.
- `finalize` : encerra o indicador de processamento.
- `Subject` / `messages$` : toasts de erro e sucesso.
- `takeUntil` : descadastro no `ngOnDestroy`.
- Interceptor HTTP: `catchError` traduz `ProblemDetails` do backend em mensagem para o usuário.

4. Quais outras bibliotecas foram utilizadas e para qual finalidade

Além do Angular 19 (Common, Forms, Router, HttpClient) e RxJS 7.8:

- Nenhuma biblioteca de UI pronta (Material, PrimeNG etc.) foi adicionada de propósito: o visual é CSS próprio, para parecer um módulo de chão de fábrica/ERP e não um template genérico.
- Fontes: IBM Plex Sans / Mono e Source Serif 4 (Google Fonts), usadas na UI e no documento de impressão.

5. Para componentes visuais, quais bibliotecas foram utilizadas

Nenhuma biblioteca de componentes visuais. Tudo é HTML e CSS próprios:

- Layout em trilho lateral + palco (shell).
- Tabelas, badges de status **Aberta** / **Fechada**.
- Botão **Imprimir nota fiscal** em destaque no editor (visível em qualquer nota escolhida).
- Overlay com *spinner* durante o processamento.
- Pré-visualização tipo documento fiscal (demonstração, não é NF-e SEFAZ).
- Toasts de erro/sucesso e indicadores de status das APIs (Estoque/Faturamento online).

6. Como foi realizado o gerenciamento de dependências no Golang (se aplicável)

Não aplicável. O backend foi escrito em **C# / .NET 10**. As dependências entram via **NuGet** nos `.csproj` e são restauradas com `dotnet restore`. Não há `go.mod`.

7. Quais frameworks foram utilizados no Golang ou C#

- ASP.NET Core Web API (controllers + Swagger/Swashbuckle)
- Entity Framework Core 10 + provedor **Npgsql** (PostgreSQL)

- `Microsoft.Extensions.Http.Resilience` (stack **Polly**: retry, timeout e circuit breaker) no cliente HTTP do faturamento para o estoque
- CORS restrito a `http://localhost:4200`

8. Como foram tratados os erros e exceções no backend

Padrão único nos dois serviços:

1. Exceções de domínio (`AppException`) com código, título, detalhe e HTTP status.
2. Middleware global converte qualquer exceção em **RFC 7807 ProblemDetails** (`title` , `detail` , `status` , `code`).
3. Casos previstos:
 - `VALIDATION` 400 — quantidade/código inválidos
 - `NOT_FOUND` 404
 - `INSUFFICIENT_STOCK` / `CONFLICT` 409 — saldo insuficiente ou nota que não está Aberta
 - `STOCK_UNAVAILABLE` 503 — estoque fora (falha real ou simulada)
 - `INTERNAL` 500 — fallback logado, sem vaziar stack para o usuário
 1. Na impressão, falha no estoque dispara **compensação** (crédito das baixas já aplicadas) e a nota **não fecha**.
 2. O frontend mostra o `detail` no toast. O usuário consegue tentar de novo.

Falha simulada: `POST /api/chaos` no estoque. O middleware passa a responder 503 em todas as rotas de negócio. `/health` e `/chaos` continuam no ar para o painel e para a restauração.

9. Caso a implementação utilize C#, indicar se foi utilizado LINQ e de que forma

Sim. Exemplos:

- Filtro e ordenação de produtos: `Where` + `OrderBy` + `Select` + `ToListAsync`
- Numeração: incremento atômico da tabela `invoice_sequences` (equivalente seguro a `Max` + 1)
- Itens da nota agrupados por produto: `GroupBy` + `Sum` das quantidades
- Insights: `Where` (abertas/fechadas), `SelectMany` dos itens, `GroupBy` da demanda e `Join` com o saldo do estoque para achar risco de impressão
- Compensação: `Enumerable.Reverse` das baixas parciais

10. Requisitos opcionais

a) Concorrência

No débito, o estoque abre transação e usa `pg_advisory_xact_lock(productId)`. Duas impressões no produto `PAR-M8` (saldo 1) não baixam o mesmo item duas vezes: uma nota fecha, a outra recebe 409 e permanece Aberta.

Script: `scripts/test-concurrency.sh`.

b) Inteligência artificial

Sem chave de LLM, para o avaliador rodar offline:

- `POST /api/ai/suggest-description` no estoque: gera descrição comercial a partir do código/nome (classificação de domínio industrial).
- `GET /api/ai/insights` no faturamento: texto sobre fila, saldo crítico e risco de impressão.
- `POST /api/ai/ask`: assistente por intenções (notas abertas, falha, concorrência) usando os dados reais.

c) Idempotência

- Impressão de nota já **Fechada** devolve sucesso sem nova baixa.
- Cada baixa leva `operationId = invoice:{id}:product:{id}`. Replay não desconta de novo.
- Header `Idempotency-Key` na impressão.
- `exhaustMap` no botão evita double-submit no browser.

11. Banco de dados

PostgreSQL 16 real (Docker, porta 5433). Bancos separados por microsserviço. As APIs executam `EnsureCreated` + seed na subida, para o avaliador não precisar rodar migration manual.

12. Como rodar a demonstração de falha

1. Crie uma nota Aberta.
2. No menu, **Simular falha do estoque**.
3. Clique em Imprimir: spinner, depois toast de indisponibilidade; status continua Aberta.
4. **Restaurar estoque**, aguarde cerca de 5 segundos (circuit breaker do Polly) e imprima de novo: a nota fecha e o saldo baixa.

Documento de detalhamento técnico pedido no enunciado do teste Korp. Entregar junto com o repositório e o vídeo.